

COMS W4156 Advanced Software Engineering (ASE)

September 24, 2024

On this day ...



The Big Bang Theory

2007 "The Big Bang Theory" created by Chuck Lorre and Bill Prady and starring [Johnny Galecki](#), [Jim Parsons](#) and [Kaley Cuoco](#) premieres on CBS

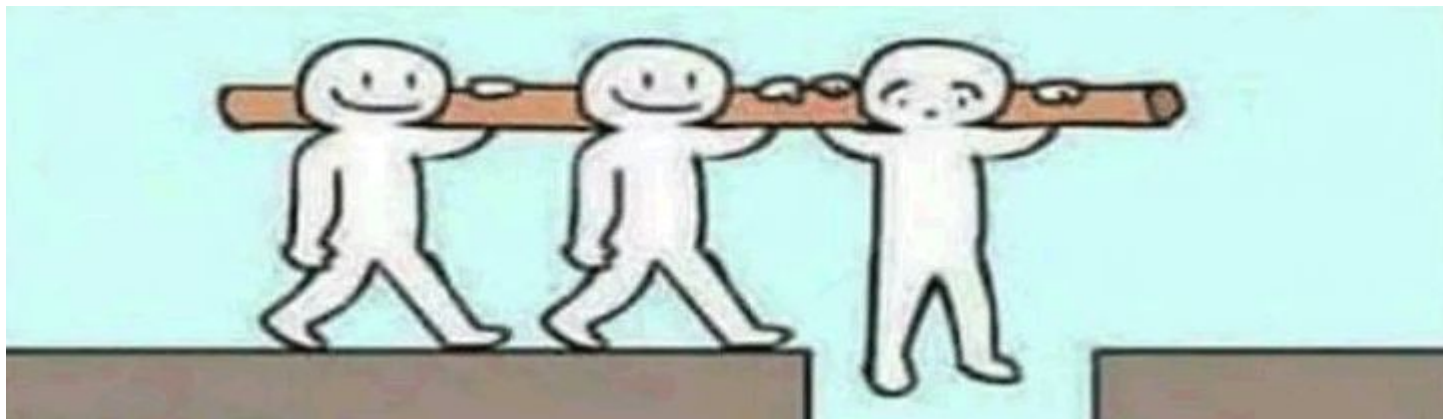


Actress

Actor and Producer

Agenda

1. **Team Formation Done?**
2. GCP deployment demo: Yixuan Li
3. More Examples of Services
4. Proposing a Service
5. Pending



If You Don't Have a Team, or a Complete Team, Speak Up



Agenda

1. Team Formation Done?
2. GCP deployment demo: Yixuan Li
3. More Examples of Services
4. Proposing a Service
5. Pending



Yixuan Li GCP Deployment Demo

[demo slides](#)



Agenda

1. Team Formation Done?
2. GCP deployment demo: Yixuan Li
3. More Examples of Services
4. Proposing a Service
5. Pending

WAYS TO SAY FOR EXAMPLE

learnenglishteam.com

- For instance
- ...As seen in...
- As you can see in the...
- Consider the example of..
- To show you what I mean
- Like
- As in...
- In other words...
- Especially
- Case in point
- By way of illustration
- Including
- In a similar case
- As proof
- Let's say
- Like this
- An example being
- Namely
- Pretend that
- Note well
- Also
- Examples include
- As seen in
- In particular
- One example is
- Particularly
- To give you an idea
- As a case in point
- Such as
- That is to say...
- These include
- Suppose that
- If you look at
- This can be seen when...
- E.g.
- This includes
- This is illustrated
- To illustrate
- Likewise
- Imagine
- As an example
- Which is made apparent when...

Lots Of Well-Known Services Used By Lots Of Clients

[PayPal Complete Payments](#)

[Google Translate API](#)

[Google Maps Platform](#)

[Meta Developers](#)

[Zoom Developers](#)

[Slack API](#)

[OpenAI Platform](#)

[Discord API](#)

[Duo Auth API](#)

[National Weather Service API](#)



You Do Not Need To Be A Big Company To Build An API

[Someone's long list of Public APIs](#)

[Someone else's long list of Public APIs](#)



Agenda

1. Team Formation Done?
2. GCP deployment demo: Yixuan Li
3. More Examples of Services
4. **Proposing a Service**
5. Pending



Next Team Project Milestones

[Team Project Proposal](#) due this Friday September 27

[Team First Iteration](#) due Friday October 18 (3 weeks to build version 1)

[Team First Iteration Demo](#) due October 25
(demo for your mentor sometime during
the week following the first iteration deadline,
your mentor must be able to "drive")



Your Project Must Target an "Acceptable User Group"

- Elderly (defined for this purpose as age 75 or over)
- Disabled or impaired (e.g., vision, hearing, speech, motor, cognitive)
- Healthcare workers (doctors, medical technicians, nurses, aides, etc.)
- First responders (police, fire, EMT)
- Indigenous peoples ("distinct social and cultural groups that share collective ancestral ties to the lands and natural resources where they live, occupy or from which they have been displaced")
- Homeless people ("lacks a fixed, regular, and adequate nighttime residence")
- Maybe others with societal benefit - ask me *in advance*



Project Requirements



1. useful for your target user group
2. maintains some relevant data persistently
3. supports multiple diverse clients
4. implemented as a service with an application programming interface (API)
5. [implicit] your team can build in three weeks
6. [implicit] you can demo your service's API without a real client (its ok to use an API testing tool as the "client")

Project Requirements Wiggle Room

1. useful for your target user group AND/OR other persons as part of their interaction with or support for your target user group
2. maintains some relevant data persistently, which could be created/updated by your clients AND/OR fetched from some external sources (your service can use existing libraries and services, including AI, but must provide added value - not just a wrapper or other passthrough)
3. supports multiple diverse clients - different user roles for the same functionality (e.g., buyers vs. sellers) do not count
4. implemented as a service with an application programming interface (API) - it must not have an end-user GUI or CLI, or any other way for end-users to talk to it directly, instead developers should write code to talk to it



Documenting Your API

If you're building an API, you want someone who is not you to use it!

Your API is the product and your API documentation is the manual, It doesn't matter how good your API is if potential adopters don't understand how to use it

Your API's documentation is the primary resource for explaining what is possible with your API and how to get started

It also serves as a place for developers to return with questions about syntax or functionality

The best API docs answer these questions
(so you don't have to)



What Is Included In API Documentation

Tells other developers what your API can do for them and why they would want to use it - what are the [use cases](#)?

May compare to alternatives, e.g., yours is better, faster, cheaper

Tells other developers how to use your API - what to do and what not to do

Covers every entry point's parameters and return values, covers authentication if applicable (are you allowed to do that?)

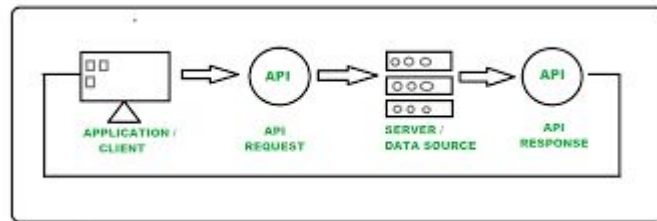
Lists all possible status or error codes it might return, explaining what they mean, what could cause this to occur, and how to handle it

Includes realistic code examples, preferably at least one for every entry point

Not intended for end-users



Testing Your API



API Testing needs to cover:

- Synchronous call/response and asynchronous events (as applicable),
- Errors visible at API interface, and
- API functions that read/write the internals of shared data or affect shared resources

Error messages (or lack thereof) are typically the last thing developers think about but the first thing their users notice when software does not work as users expected

This includes developers who are users of APIs. But error messages need to be appropriate for code to code not necessarily display to humans - e.g., status codes instead of long strings (but documentation needs to explain each status code with human-readable text)

For Each API Entry point (aka Endpoint)

Provide inputs intended to force 'happy path' (or paths) to make sure basic functionality works. [Happy Path](#) = typical valid input where (most likely) nothing can go wrong

Then try to call the entry point without a mandatory parameter, a parameter of the wrong type, and parameters out of order

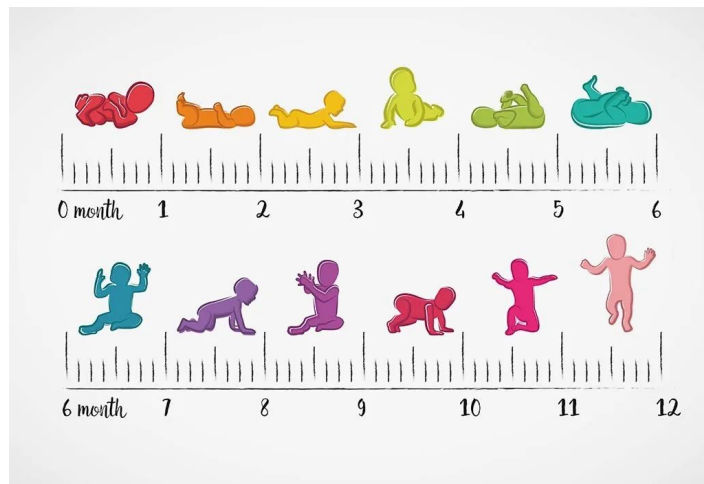


Devise inputs to try to force every status code that the entry point could return. If the caller is supposed to “do something” for a status code that affects the API, test with and without actually doing it (do something else instead) - *more on choosing inputs coming later*

If the caller needs to supply callback code for API events, test with mock callers that do and do not actually fulfill the expected interface, and do / do not actually perform any expected actions that affect the API

Agenda

1. Team Formation Done?
2. GCP deployment demo: Yixuan Li
3. More Examples of Services
4. Proposing a Service
5. Pending



Last Miniproject Deadline

[Part Individual Miniproject part 3](#) due this Friday September 27

There should be a note with your I1 submission (courseworks comment) saying which Mentor to add to your repo as a collaborator, do it!



Thursday

Finish anything we didn't get to today

Pitch Project Proposal Ideas

If time permits, code review



Ask Me Anything



BE REASONABLE